Tutorial 4: Data Augmentation

Name: Laiba Niazi

Roll Number: 536672

1. Objective

The primary objective of this tutorial is to implement a data augmentation pipeline using PyTorch's torchvision.transforms and to define a basic feed-forward neural network architecture for image classification. Data augmentation is used to artificially increase the size of a dataset by creating modified versions of images, which helps improve model generalization.

2. Environment Setup

The implementation utilizes the following key Python libraries:

- torch: Main library for tensor operations and neural networks.
- torchvision: Used for image transformations and utility functions like save_image.
- PIL (Pillow): For image loading.
- matplotlib: For visualizing the results within the notebook.

3. Data Augmentation Pipeline

The transformation pipeline was designed to introduce significant variability into the training data. The following transformations were applied in sequence:

- **Random Rotation:** Images are randomly rotated within a range of 40 degrees.
- **Random Affine (Shear):** Applies a shear transformation of up to 15 degrees.
- **Random Resized Crop:** Randomly crops the image and resizes it to 224x224 pixels with a scale factor between 0.8 and 1.2.
- **Horizontal Flip:** Randomly flips the image horizontally with a 50% probability.
- **Color Jitter:** Adjusts the brightness of the image between 0.5 and 1.5 of its original value.

```python
# Define the transformation pipeline to introduce variability
augmentation_transforms = transforms.Compose([
    transforms.RandomRotation(degrees=40),              # rotation_range=40
    transforms.RandomAffine(degrees=0, shear=15),       # shear_range=0.2 (approx 15 deg)
    transforms.RandomResizedCrop(size=(224, 224),scale=(0.8, 1.2)),     # zoom_range=0.2
    transforms.RandomHorizontalFlip(p=0.5),             # horizontal_flip=True
    transforms.ColorJitter(brightness=(0.5, 1.5)),      # brightness_range=(0.5, 1.5)
    transforms.ToTensor()                               # Convert to Tensor for saving
])
```

[6] ✓ 0.0s Python

4. Neural Network Architecture

A Sequential model was constructed to process 28x28 grayscale images (flattened to 784 features). The architecture consists of the following layers:

1. **Flatten Layer:** Reshapes the input into a 1D tensor.
2. **Linear Layer 1:** 784 input features to 128 hidden neurons.
3. **ReLU Activation:** Introduces non-linearity.
4. **Linear Layer 2:** 128 input features to 64 hidden neurons.
5. **ReLU Activation.**
6. **Output Layer:** 64 input features to 10 output neurons (representing 10 digit classes).
7. **LogSoftmax:** Activation function for multi-class classification.

```python
# Build the neural network model
model = nn.Sequential(
    nn.Flatten(),                          # Flatten 28x28 images to 784 features
    nn.Linear(28 * 28, 128),               # Hidden Layer with 128 neurons
    nn.ReLU(),                             # ReLU activation
    nn.Linear(128, 64),                    # Hidden Layer with 64 neurons
    nn.ReLU(),                             # ReLU activation
    nn.Linear(64, 10),                     # Output layer with 10 neurons
    nn.LogSoftmax(dim=1)                   # Softmax activation for 10 digits
)

print(model) # Display architecture
```

[7] ✓ 0.0s Python

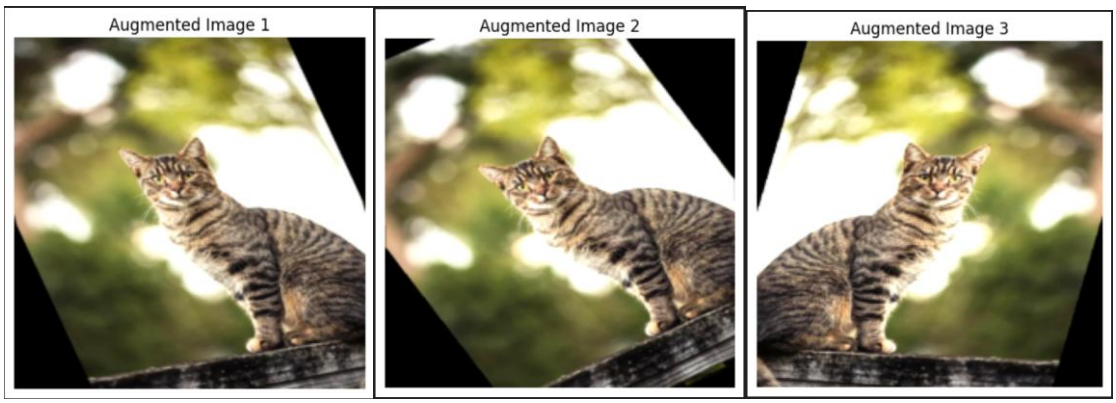
```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=128, bias=True)
  (2): ReLU()
  (3): Linear(in_features=128, out_features=64, bias=True)
  (4): ReLU()
  (5): Linear(in_features=64, out_features=10, bias=True)
  (6): LogSoftmax(dim=1)
)
```

5. Results and Visualization

The script processes an input image (named Image.jpg) and generates 40 unique augmented versions stored in the PyTorch_Augmented_Images directory.

5.1 Visualization of Augmented Samples

The first few augmented images generated by the pipeline are displayed below to demonstrate the effects of rotation, cropping, and color jittering.



5.2 File System Output

After execution, the program confirms the successful saving of all 40 images to the local directory.

```
...   Success! 40 images saved in: C:\Users\M.T\Desktop\NUST\2nd Sem\DL\DLTutorials\PyTorch_Augmented_Images
```

6. Conclusion

By utilizing the transforms module in PyTorch, we successfully created a robust data augmentation strategy. This method ensures that the neural network sees a different variation of each image during every training epoch, thereby reducing overfitting and increasing the accuracy of the model on unseen data. The defined architecture provides a solid baseline for classification tasks involving small-scale image data.